



School of Future Tech

Case Study Report

on

Case Study - 173

Music Playlist Manager (Doubly Linked List)

Danish Shaikh

150096725147

B.Tech CSE (2025-29)

Index

1. Introduction to the Case Study
2. Problem Statement / Case Background
3. System Architecture and Design
4. Complexity Analysis
5. Implementation Snapshots
6. Conclusion

1. Introduction to the Case Study

TerminalTunes is a terminal-based C++ application that simulates a real-world music playlist management system. Developed for the Data Structures and Algorithms - I subject at ITM Skills University, School of Future Tech, the application uses advanced ANSI formatting to deliver an interactive UI while relying strictly on custom data structures without external libraries.

Problem Statement

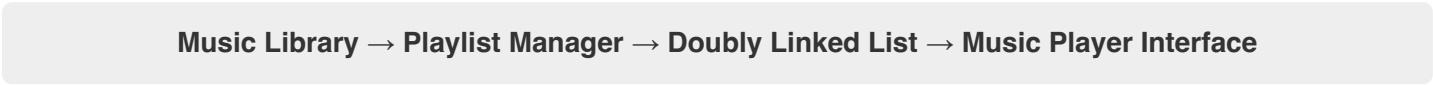
Industry Context: Music streaming applications allow users to create playlists, rearrange songs, and move between tracks efficiently.

Workflow: Songs are stored as nodes containing previous and next references. Users can move to the next song, return to previous songs, insert new tracks, or remove unwanted tracks.

Example Scenario: A playlist contains Song A, Song B, and Song C. The user listens to Song B and decides to return to Song A using the previous pointer. The system provides smooth navigation, dynamic playlist management, enhanced user experience, and efficient music playback.

2. System Architecture and Design

The system is built upon a modular architecture in C++17:



- **Doubly Linked List:** Used as the core Playlist. Allows $O(1)$ bidirectional traversal.
- **Custom Stack (SLL):** Used for tracking the listening history (LIFO).
- **Custom Queue (SLL):** Used for the upcoming song queue (FIFO).
- **Fisher-Yates Algorithm:** Used for in-place shuffling of the playlist nodes.

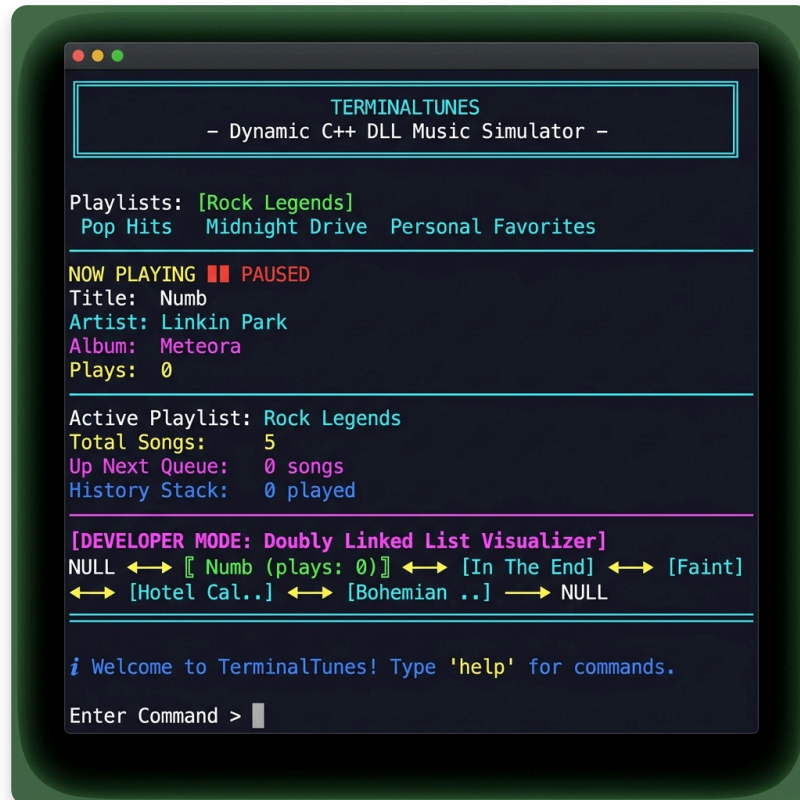
3. Complexity Analysis

Operation	Data Structure	Time Complexity	Space Complexity
Play Next/Prev Song	Doubly Linked List	$O(1)$	$O(1)$
Insert at Beginning/End	Doubly Linked List	$O(1)$	$O(1)$
Search Song by Title	Doubly Linked List	$O(n)$	$O(1)$
Shuffle Playlist	Vector + DLL Rebuild	$O(n)$	$O(n)$
Push to / Pop History	Custom Stack (SLL)	$O(1)$	$O(1)$

4. Implementation Snapshots

Main Dashboard and Visualizer

The main UI displays the current song, active queues, and a real-time developer visualizer for the Doubly Linked List pointers.



Statistics Dashboard

Calculates metrics like total plays, unique artists, and most played track using traversal algorithms.



5. Conclusion

TerminalTunes successfully meets all objectives by delivering a fully functional, memory-safe playlist simulator. By strictly using C++ and custom-built data structures (Doubly Linked Lists, Stacks, Queues), the project serves as a comprehensive demonstration of memory management, pointer manipulation, and Object-Oriented design. The system runs optimally with $O(1)$ time complexity for core playback operations.

6. References

1. Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.
2. ITM Skills University — Data Structures and Algorithms Course Notes, School of Future Tech, 2025.
3. GitHub Repository — <https://github.com/Danish1323/dsa-sem2-project>